P1664R3
reconstructible_range - a concept for putting ranges back together

Published Proposal, 2021-04-15

Authors:

JeanHeyd Meneide

Hannes Hauswedell

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++23

Latest:

https://thephd.github.io/_vendor/future_cxx/papers/d1664.html

---

Abstract

This paper proposes new concepts to the Standard Library for ranges called Reconstructible Ranges for the purpose of ensuring a range/view broken down into its two iterators can be "glued" back together using an ADL-found function taking a tag, the range's iterator, and the range's sentinel.

Table of Contents

§ 1. Revision History

§ 1.1. Revision 3 - April 15th, 2021

- Vastly improve examples.
- Re-target motivation.
- Adjust fixes for things that have changed since C++20.

§ 1.2. Revision 2 - January 13th, 2020

- Improve wording and add more explanation in the design sections for the changes.

§ 1.3. Revision 1 - November 24th, 2019

- Change to `snake_case` concepts. (November 6th)
- Update wording to target [n4835]. (November 6th)
- Moved by LWG! 🎉 (November 6th)
- Last minute objection to this paper in plenary.
- Withdrawn; targeting C++23 instead with better design.

- Explored 3 different designs offline: settle on 1 (thanks, Oktal).

§ 1.4. Revision 0 - August, 5th, 2019

- Initial release.

§ 2. Motivation

C++ 20

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector<int> vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
span<int> v = s | views::drop(1) | views::take(10)
           | views::drop(1) | views::take(10);
```

❌ - Compilation error

```
template <typename T>
using span = quickcppl

std::vector<int> vec{1
span<int> s{vec.data()
span<int> v = s | view
           | views::dr
```

✔️ - Compiles and works with r

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
auto v = s | views::drop(1) | views::take(10)
           | views::drop(1) | views::take(10);
```

⚠️ - Compiles, but `decltype(v)` is `ranges::take_view<ranges::drop_view<ranges::take_view<ranges::drop_view<span<int, dynamic_extent>>>>>`

```
template <typename T>
using span = quickcppl

std::vector vec{1, 2,
span<int> s{vec.data()
auto v = s | views::dr
           | views::dr
```

✔️ - Compiles and works with r `quickcpplib::span<int>`

```
std::u8string name = "AYLEMI·MMETELIM ʒʊʒ ·ɩʊɩʊɭ ·ꞆƧ";
char16_t conversion_buffer[432];

std::u8string_view name_view(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode(input, output);
// compiler error here !!
std::u8string_view unprocessed_code_units
        = encoding_result.input;
std::span<char16_t> unconsumed_output
        = encoding_result.output;
```

❌ - Compilation error

```
std::u8string name = "
char16_t conversion_bu

std::u8string_view nam
std::span<char16_t> ou

auto encoding_result =
// compiler error here
std::u8string_view unp
        = encoding_res
std::span<char16_t> un
        = encoding_res
```

✔️ - Compiles and works, types

```
std::u8string name = "AYLEMI·MMETELIM ʒʊʒ ·ɩʊɩʊɭ ·ꞆƧ";
char16_t conversion_buffer[432];

std::u8string_view name_view(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode(input, output);
auto unprocessed_code_units = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

⚠️ - Compiles, but `decltype(unprocessed_code_units)` is `ranges::subrange<std::u8string_view::iterator, std::u8string_view::iterator>` and `decltype(unconsumed_output)` is `ranges::subrange<std::span<char16_t,`

```
std::u8string name = "
char16_t conversion_bu

std::u8string_view nam
std::span<char16_t> ou

auto encoding_result =
auto unprocessed_code_
auto unconsumed_output
```

✔️ - Compiles and works, wher `std::u8string_view` and de `std::span<char16_t>`.

`std::dynamic_extent>::iterator, std::span<char16_t, std::dynamic_extent>::iterator>`

Currently in C++, there is no Generic ("with a capital G") way to take a range apart with its iterators and put it back together. That is, the following code is not guaranteed to work:

```
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
        using uRange = std::remove_cvref_t<Range>;
        if (std::ranges::empty(range)) {
                // ⛔!
                // ... the below errors
                return uRange(std::forward<Range>(range));
        }
        /* perform some work with the
        iterators or similar */
        auto first = std::ranges::begin(range);
        auto last = std::ranges::end(range);
        if (*first == u'\0xEF') {
                // ...
                std::advance(first, 3);
                // ...
        }
        // ... algorithm finished,
        // return the "updated" range!

        // ⛔!
        // ... but the below errors
        return uRange(std::move(first), std::move(last));
}

int main () {
        std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
        // this line will error
        std::string_view sub_view
                = operate_on_and_return_updated_range(meow_view);
        return 0;
}
```

The current fix is to employ `std::ranges::subrange<I, S, K>` to return a generic subrange:

```
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
        using uRange = std::remove_cvref_t<Range>;
        using I = std::Iterator<uRange>;
        using S = std::Sentinel<uRange>;
        using Result = std::ranges::subrange<I, S>;
        if (std::ranges::empty(range)) {
                return Result(std::forward<Range>(range));
        }
        // perform some work with the
        // iterators or similar
        auto first = std::ranges::begin(range);
        auto last = std::ranges::end(range);
        if (*first == u'\0xEF') {
                // ...
                std::advance(first, 3);
                // ...
        }
        // ... algorithm finished,
        // return the "updated" range!

        // now it works!
```

```
        return Result(std::move(first), std::move(last));
}

int main () {
        std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
        auto sub_view
                = operate_on_and_return_updated_range(meow_view);
        // decltype(sub_view) ==
        //   std::ranges::subrange<std::string_view::iterator,std::string_view::iterator>
        //   which is nowhere close to ideal.
        return 0;
}
```

This makes it work with any two pair of iterators, but quickly becomes undesirable from an interface point of view. If a user passes in a `quickcpplib::span<T, Extent>` or a `llvm::StringRef` that interface and information is entirely lost to the user of the above function. `std::ranges::subrange<Iterator, Sentinel, Kind>` does not -- and cannot/should not -- mimic the interface of the view it was created from other than what information comes from its iterators: it is the barebones idea of a pair-of-iterators/iterator-sentinel style of range. This is useful in the generic sense that if a library developer must work with iterators, they can always rely on creation of a `std::ranges::subrange` of the iterator and sentinel.

§ 2.1. Preservation of Properties

Unfortunately, always using `std::ranges::subrange` decreases usability for end users. Users who have, for example a `llvm::StringRef`, would prefer to have the same type after calling an algorithm. These types have functions and code that are purpose-made for `string_view`-like code: losing that intent means needing to reconstruct that from the ground up all over again. There is little reason why the original type needs to be discarded if it supports being put back together from its iterators.

The break-it-apart-and-then-generic-`subrange` approach also discards any range-specific storage optimizations and layout considerations, leaving us with the most bland kind of range similar to the "pair of iterators" model. Consider, for example, `llfio::span<T>`. The author of the Low Level File IO (LLFIO) library, Niall Douglass, has spent countless days submitting pull requests and change requests to MSVC, GCC, and Clang's standard libraries to ask them to change their structural layout to match things like the `iovec` of their platform or their asynchronous buffer span types. This optimization matters because it realizes the performance difference between having to individually transformed a container of `span`s to a more suitable type, or being able to simply `memcpy` the desired sequences of spans into the underlying asynchronous operations. If range operations were to be performed on the `llfio::span<T>` type, a `std::ranges::subrange` would hold, in most cases, two iterators rather than an iterator and a size. This completely eliminates the `memcpy` optimization. This is not the only place this happens, however: other types have different storages requirements that are not appropriately captured by their iterators in the most general sense, but may benefit from reconstruction and desired type information for the target range they should be constructed into.

§ 2.2. Compile-Time and Interoperability

Compilation time goes up as well in the current paradigm. Users must spawn a fresh `std::ranges::subrange<I, S, K>` for every different set of iterator/sentinel/kind triplet, or handle deeply nested templates in templates as the input types. This makes it impossible to compile interfaces as dynamic libraries without having to explicitly materialize or manually cajole a `std::ranges::subrange` into something more palatable for the regular world.

There is also a problem where there are a wide variety of ranges that could conceivably meet this criterion, but do not. The author of this paper was not the only one to see utility for this. [p1739r4] does much the same that this paper does, without the introduction of a concept to formalize the behavior it presents. In particular, it selects views which can realistically have their return types changed to match the input range and operations being performed (or a similarly powerful alternative) by asking whether they can be called with a function called `reconstruct` from a subrange of the iterators with the expressions acted upon.

- Ranges should be reconstructible from their iterators (or subrange of their iterators) where applicable;

- and, reconstructible ranges serve a useful purpose in generic algorithms, including not losing information and returning it in a much more cromulent and desirable form.

Therefore, this paper formalizes that concept and provides an **opt-in, user-overridable way** to return a related "reconstructed" type from an tag, an iterator, and a sentinel.

§ 3. Design

The design is given in 2 concepts added to the standard:

```
template <class R,
        class It = ranges::iterator_t<remove_reference_t<R>>,
        class Sen = ranges::sentinel_t<remove_reference_t<R>>>
concept pair_reconstructible_range =
        ranges::range<R> &&
        ranges::borrowed_range<remove_reference_t<R>> &&
        requires (It first, Sen last) {
                reconstruct(
                        in_place_type<remove_cvref_t<R>>,
                        std::move(first),
                        std::move(last)
                );
        };

template <class R,
        class Range = remove_reference_t<R>>
concept reconstructible_range =
        ranges::range<R> &&
        ranges::borrowed_range<remove_reference_t<R>> &&
        requires (Range first_last) {
                reconstruct(
                        in_place_type<remove_cvref_t<R>>,
                        std::move(first_last)
                );
        };
```

It is the formalization that a range can be reconstructed from its begin iterator and end iterator/sentinel. It also provides a concept for allowing a range to be put back together from another range, which may be useful for some types of ranges where more than the iterator and sentinel are required. This allows a developer to propagate the input type's properties after modifying its iterators for some underlying work, algorithm or other effect. This concept is also the basis of the idea behind [p1739r4], which was accepted in a lesser form for C++20 with the intent of this paper following up on it.

Both concepts require that the type with any references removed model the concept `borrowed_range`. This ensures that the validity of the iterators is in fact independent of the lifetime of the range they originate from and that a "reconstructed" range does not depend on the original. We remove reference before performing this check, because all reference types that model `Range` also model `borrowed_range` and the intent of the proposed changes is narrower: (re)construction is assumed to be in constant time (this typically implies that `R` also models `View`, but it is sufficient to check `borrowed_range<remove_reference_t<R>>`).

Note that this explicitly excludes types like `std::vector<int> const &` from being reconstructible.

§ 3.1. Multiple, Safe Reconstruction Points

Consider a hypothetical `c_string_view` type. It is impossible to, from normal `const char*` iterator pair ascertain the range is null-terminated and is thus a "C string". Therefore, a reconstruct for `c_string_view` would not return a `c_string_view`, but a normal `string_view`.

On the other hand, if there was a `c_string_sentinel` sentinel with a `const char*` iterator that was preserved through a series of actions and algorithms, that kind of information would allow us to return a `c_string_view` from a reconstruction operation.

The `reconstruct` extension point supports both of these scenarios:

```
class c_string_view {
        /* blah... */

        // reconstruct: with [const char*, const char*) iterators
        // no guarantee it's null terminated: decay to string_view
        friend constexpr std::string_view reconstruct (
                std::in_place_type_t<c_string_view>,
                const char* first, const char* last) {
                return std::string_view(first, last);
        }

        // reconstruct: with static [const char*, c_string_sentinel)
        // static guarantee by type: return a c_string_view
        friend constexpr c_string_view reconstruct (
                std::in_place_type_t<c_string_view>,
                const char* first, c_string_sentinel) {
                return c_string_view(first);
        }
};
```

This is a level of flexibility that is better than the Revision 0 design, and can aid in providing better static guarantees while decaying gracefully in other situations.

§ 3.2. Should this apply to all Ranges?

Not all ranges can meet this requirement. Some ranges contain state which cannot be trivially propagated into the iterators, or state that cannot be reconstructed from the iterator/sentinel pair itself. However, most of the common ranges representing unbounded views, empty views, iterations viewing some section of non-owned storage, or similar can all be reconstructed from their iterator/iterator or iterator/sentinel pair.

For example `std::ranges::single_view` contains a exposition *semiregular-box* template type (ranges.semi.wrap) which holds a value to iterate over. It would not be possible to reconstruct the exact same range (e.g., iterators pointing to the exact same object) with the semi-regular wrapper.

Finally, there are ranges which could be reconstructible by just the definition of a constructor that takes iterators or a `subrange`. While this was a benefit in that the majority of conforming ranges were reconstructible, it made a situation where someone could write a strange range. For example, consider this alternative implementation of `rng | views::drop(1)`:

```
class pop_front_view {
private:
        int *begin_, *end_;

public:
        pop_front_view() = default;

        pop_front_view(int* begin, int* end)
                : begin_(begin==end?begin:begin+1), end_(end)
        {}

        int* begin() const { return begin_; }

        int* end() const { return end_; }
};
```

Reconstructing this range using the constructors is a surefire way to have a developer scratching at their head, wondering what is going on. Therefore, rather than require reconstruction through the constructor, we rely instead on an Customization Point Object design instead.

3.3. Applicability

There are many ranges to which this is applicable, but only a handful in the standard library need or satisfy this. If [p1391r2] and [p1394r2] are accepted (they were), then the two most important view types -- `std::span<T, Extent>` and `std::basic_string_view<Char, Traits>` -- will model the _spirit_ of the concept (but lack the customization point to make it generic). `std::ranges::subrange<Iterator, Sentinel, Kind>` already fits this as well. By formalizing concepts in the standard, we can dependably and reliably assert that these properties continue to hold for these ranges. We intend to add a `constexpr friend reconstruct` function to all of the non-`ranges::subrange` types.

There are also upcoming ranges from [range-v3] and elsewhere that could model this concept:

- [p1255r4]'s `std::ranges::ref_maybe_view`;
- [p0009r9]'s `std::mdspan`;
- and, soon to be proposed by this author for the purposes of output range algorithms, [range-v3]'s `ranges::unbounded_view`.

And there are further range adaptor closure objects that could make use of this concept:

- `views::slice`, `views::take_exactly`, `views::drop_exactly` and `views::take_last` from [range-v3]

Note that these changes will greatly aid other algorithm writers who want to preserve the same input ranges. In the future, the standard may provide an `ranges::reconstruct(...)` algorithm for these types.

3.4. Two Concepts

By giving these ranges `Iterator, Sentinel`, and/or `Range` reconstruction functions, we can enable a greater degree of interface fidelity without having to resort to `std::ranges::subrange` for all generic algorithms or similar input/output iterator holders as we have in much of the current `ranges <algorithms>` library.

`reconstruct(std::inplace_type<R>, Iterator, Sentinel)` reconstruction is important for building `llvm::StringRef`, `llfio::span`, `kokos::mdspan`, and other types that go into algorithms.

because one-argument functions can have extremely overloaded meanings. It also produces less compiler boilerplate to achieve the same result of reconstructing the range when one does not have to go through `std::ranges::subrange<I, S, K>`. However, it is important to attempt to move away from the iterator, sentinel model being deployed all the time: `std::ranges::subrange` offers a single type that can accurately represent the intent and can be fairly easy to constrain overload sets on (most of the time) due to being a single family of types (anyone can check for a the type being a specialization of `ranges::subrange`, whereas iterators span multiple different families of types).

This paper includes two concepts that cover both reconstructible methods.

4. Deployment Experience

This change was motivated by Hannes Hauswedell's [p1739r4] and became very important for the ranges work done with text encoding. There is a C++17 implementation at the soasis/text repository here, which is mimicking the interface needed for [p1629r1].

5. Impact

As a feature that is opt-in thanks to the `ranges::reconstruct` Customization Point Design, no currently created range or previously made range is permanently affected.

Furthermore, this is a new and separate concept. It is not to be added to the base `Range` concept, or added to any other concept. It is to be applied separately to the types which can reasonably support it for the benefit of algorithms and code which can enhance the quality of their implementation.

Finally, Hannes Hauswedell's [p1739r4] with the explicit intention to mark certain ranges as reconstructible by hardcoding their behavior into the standard and come back with an opt-in fix during the C++23 cycle. This paper completes that promise.

§ 6. Proposed Changes

The following wording is relative to the latest C++ Draft paper.

§ 6.1. Feature Test Macro

This paper results in a concept to help guide the further development of standard ranges and simplify their usages in generic contexts. There is one proposed feature test macro, `__cpp_lib_reconstructible_range`, which is to be input into the standard and then explicitly updated every time a `reconstruct` from a is added to reflect the new wording. We hope that by putting this in the standard early, most incoming ranges will be checked for compatibility with `pair_reconstructible_range` and `reconstructible_range`.

§ 6.2. Intent

The intent of this wording is to provide greater generic coding guarantees and optimizations by allowing for a class of ranges and views that model the new exposition-only definitions of a reconstructible range:

- add a new feature test macro for reconstructible ranges to cover constructor changes;
- add a new customization point object for `ranges::reconstruct`,
- and, add two new concepts to [range.req].

If *borrowed_range* is changed to the `borrowed_range` concept name, then this entire proposal will rename all its uses of *borrowed_range*.

For ease of reading, the necessary portions of other proposal's wording is duplicated here, with the changes necessary for the application of reconstructible range concepts. Such sections are clearly marked.

§ 6.3. Proposed Library Wording

Add a feature test macro `__cpp_lib_reconstructible_range`.

Insert into §24.2 Header `<ranges>` Synopsis [ranges.syn] a new customization point object in the inline namespace:

```
namespace std::ranges {
inline namespace unspecified {
…

nline constexpr nspecified reconstruct = unspecified;

…

}
}
```

Insert into §24.4.2 Ranges [range.range]'s after paragraph 7, one additional paragraph:

> [8] The concepts `pair_reconstructible_range` and `reconstructible_range` concepts describe the requirements on ranges that are efficiently constructible from values of their iterator and sentinel types.
>
> ```
> template <class R,
>         class It = ranges::iterator_t<remove_reference_t<R>>,
>         class Sen = ranges::sentinel_t<remove_reference_t<R>>>
> concept pair_reconstructible_range =
>         ranges::range<R> &&
>         ranges::borrowed_range<remove_reference_t<R>> &&
>         requires (It first, Sen last) {
>                 reconstruct(
>                         in_place_type<remove_cvref_t<R>>,
>                         std::move(first),
>                         std::move(last)
>                 );
>         };
>
> template <class R, class Range = remove_reference_t<R>>
> concept reconstructible_range =
>         ranges::range<R> &&
>         ranges::borrowed_range<remove_reference_t<R>> &&
>         requires (Range first_last) {
>                 reconstruct(
>                         in_place_type<remove_cvref_t<R>>,
>                         std::move(first_last)
>                 );
>         };
> ```
>
> [9] Let `r` be a range with type `R`.
>
> > [9.1] — Let `re` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<R>>, ranges::begin(r), ranges::end(r))` if such an expression is well-formed. `r` models `ranges::pair_reconstructible_range` if
> >
> > — `ranges::begin(r) == ranges::begin(re)` is true, and
> >
> > — `ranges::end(r) == ranges::end(re)` is true.
> >
> > [9.2] — Let `sub_re` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<R>>, std::move(r))`, if such an expression is well-formed. Then `sub_re` models `ranges::reconstructible_range` if
> >
> > — `ranges::begin(r) == ranges::begin(sub_re)` is true, and
> >
> > — `ranges::end(r) == ranges::end(sub_re)` is true.

Insert a new sub-clause "§24.3.13 `ranges::reconstruct` [**range.prim.recons**]", after "§24.3.12 `ranges::cdata` [**range.prim.cdata**]":

24.3.12 `ranges::reconstruct` [range.prim.recons]

[1] The name `reconstruct` denotes a customization point object.

[2] The expression `ranges::reconstruct(in_place_type<R>, I, S)` for some type R and some sub-expressions I and S is expression-equivalent to:

(2.1) `reconstruct(in_place_type<R>, std::move(I), std::move(S))` if it is a valid expression and R, `decltype(I)`, and `decltype(S)` model `pair_reconstructible_range`.

(2.2) Otherwise, `reconstruct(in_place_type<R>, ranges::subrange<remove_cvref_t<decltype(I)>, remove_cvref_t<decltype(S)>>(std::move(I), std::move(S)))` if it is a valid expression and R, `decltype(I)`, and `decltype(S)` model `reconstructible_range`.

(2.3) `ranges::subrange<remove_cvref_t<decltype(I)>, remove_cvref_t<decltype(S)>>(std::move(I), std::move(S))` if it is a valid expression.

(2.4) Otherwise, `ranges::reconstruct(std::in_place_type<R>, I, S)` is ill-formed.

[3] Let SR be some sub-expression. The expression `ranges::reconstruct(in_place_type<R>, SR)` for some type R and sub-expression SR is expression-equivalent to:

(2.1) `reconstruct(in_place_type<R>, std::move(SR))` if it is a valid expression, and R and `decltype(SR)` model `reconstructible_range`.

(2.2) Otherwise, `reconstruct(in_place_type<R>, ranges::begin(std::move(SR)), ranges::end(std::move(SR)))` if it is a valid expression and R, `range::iterator_t<decltype(SR)>`, and `range::sentinel_t<decltype(SR)>` model `pair_reconstructible_range`.

(2.3) Otherwise, `return SR`.

Add to "§21.4.3.1 `General`" [string.view.template.general] a `friend` function ADL extension point for `basic_string_view`:

```
// [string.view.reconstruct]
friend constexpr basic_string_view
reconstruct(const_iterator first, const_iterator last) noexcept;
```

Add a new subclause "§21.4.??? `Range Reconstruction`" [string.view.reconstruct] in §21.4:

21.4.??? Range Reconstruction [string.view.reconstruct]

```
friend constexpr basic_string_view
reconstruct(const_iterator first, const_iterator last) noexcept;
```

[1] Returns: `basic_string(std::move(first), std::move(last))`.

Add to "§22.7.3.1 `Overview`" [span.overview] a `friend` function ADL extension point for `span`:

```
// [span.reconstruct]
template <class It, class End>
        friend constexpr auto
        reconstruct(It first, End last) noexcept;
```

Add a new subclause "§22.7.3.???? `Range Reconstruction`" [span.reconstruct] in §22.7.3:

22.7.3.☐☐☐☐ Range Reconstruction [**span.reconstruct**]

```
template <class It, class End>
        friend constexpr auto
        reconstruct(const_iterator first, const_iterator last) noexcept;
```

[1] Constraints: Let `U` be `remove_reference_t<iter_reference_t<It>>`.

(1.1) — `is_convertible_v<U(*)[], element_type(*)[]>` is `true`. [*Note 2*: The intent is to allow only qualification conversions of the iterator reference type to element_type. — *end note*]

(1.2) — `It` satisfies `contiguous_iterator`.

(1.3) — `End` satisfies `sized_sentinel_for<It>`.

(1.4) — `is_convertible_v<End, size_t>` is `false`.

[2] Let `MaybeExtent` be an implementation-defined constant expression of type `size_t`. If it is not equal to `dynamic_extent`, then `MaybeExtent` shall be a value equal to `distance(first, last)`.

[3] Returns: `span<ElementType, MaybeExtent>(std::move(first), std::move(last))`.

[4] Remarks: the return type can be promoted to a `span` with a static extent if the implementation is capable of deriving such information from the given iterator and sentinel.

Add to "§24.5.4.1 `General`" [range.subrange.general] a `friend` function ADL extension point for `subrange`:

```
template <class It, class End>
        friend constexpr auto
        reconstruct(It first, End last) noexcept;
```

Add a new subclause "§24.5.4.☐☐☐☐ `Range Reconstruction`" [range.subrange.reconstruct] in §24.5.4:

22.7.3.☐☐☐☐ Range Reconstruction [**range.subrange.reconstruct**]

```
friend constexpr subrange
reconstruct(I first, S last) noexcept;
```

[1] Returns: `subrange(std::move(first), std::move(last))`.

Add to "§24.6.4.2 Class template `iota_view`" [range.iota] a `friend` function ADL extension point for `iota_view`:

```
friend constexpr iota_view
reconstruct(iterator first, sentinel last) noexcept;
```

Add one new paragraph to "§24.6.4.2 Class template `iota_view`" [range.iota]:

```
friend constexpr iota_view
reconstruct(iterator first, sentinel last) noexcept;
```

☐☐ Returns: `iota_view(std::move(first), std::move(last))`.

Add to "§24.6.2.2 Class template `empty_view`" [range.empty.view] a `friend` function ADL extension point for `empty_view`:

```
        friend constexpr empty_view
        reconstruct(iterator, sentinel) noexcept {
                return empty_view();
        }
```

Modify "§24.7.7.1 Overview " [range.take.overview] for `views::take`'s paragraph 2:

> [2] The name `views::take` denotes a range adaptor object ([range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression views::take(E, F) is expression-equivalent to:
>
> ~~(2.1) — If `T` is a specialization of `ranges::empty_view` ([range.empty.view]), then `((void) F, decay-copy(E))`.~~
>
> ~~(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is~~
>
> ~~(2.2.1) — a specialization of `span` ([views.span]) where `T::extent == dynamic_extent`,~~
>
> ~~(2.2.2) — a specialization of `basic_string_view` ([string.view]),~~
>
> ~~(2.2.3) — a specialization of `ranges::iota_view` ([range.iota.view]), or~~
>
> ~~(2.2.4) — a specialization of `ranges::subrange` ([range.subrange]),~~
>
> ~~then `T{ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F)}`, except that `E` is evaluated only once.~~
>
> ~~(2.3) — Otherwise, `ranges::take_view{E, F}`.~~
>
> (2.1) — If `T` is a specialization of `ranges::empty_view` ([range.empty.view]), then `((void) F, decay-copy(E))`.
>
> (2.2) — Otherwise, if `T` models `random_access_range`, `sized_range`, and `reconstructible_range`, then `T{ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F)}`, except that `E` is evaluated only once.
>
> (2.3) — Otherwise, `ranges::take_view{E, F}`.

Modify "§24.7.9.1 Overview " [range.take.overview] for `views::drop`'s paragraph 2:

[2] The name `views::drop` denotes a range adaptor object ([range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::drop(E, F)` is ill-formed. Otherwise, the expression `views::drop(E, F)` is expression-equivalent to:

~~(2.1) — If `T` is a specialization of `ranges::empty_view` ([range.empty.view]), then `((void) F, decay-copy(E))`.~~

~~(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is~~

~~(2.2.1) — a specialization of `span` ([views.span]) where `T::extent == dynamic_extent`,~~

~~(2.2.2) — a specialization of `basic_string_view` ([string.view]),~~

~~(2.2.3) — a specialization of `ranges::iota_view` ([range.iota.view]), or~~

~~(2.2.4) — a specialization of `ranges::subrange` ([range.subrange]),~~

~~then `T{ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E)}`, except that `E` is evaluated only once.~~

~~(2.3) — Otherwise, `ranges::drop_view{E, F}`.~~

(2.1) — If `T` is a specialization of `ranges::empty_view` ([range.empty.view]), then `((void) F, decay-copy(E))`.

(2.2) — Otherwise, if `T` models `random_access_range`, `sized_range`, and `reconstructible_range`, then `T{ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E)}`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::drop_view{E, F}`.

§ 7. Acknowledgements

Thanks to Corentin Jabot, Christopher DiBella, and Hannes Hauswedell for pointing me to [p1035r7] and [p1739r4] to review both papers and combine some of their ideas in here. Thanks to Eric Niebler for prompting me to think of the generic, scalable solution to this problem rather than working on one-off fixes for individuals views.

Thank you to Oktal, Anointed of ADL, Blessed Among Us, and Morwenn, the ever-watching Code Guardian for suggesting improvements to the current concept form.

§ References

§ Informative References

[N4835]
Richard Smith. Working Draft, Standard for Programming Language C++. 8 October 2019. URL: https://wg21.link/n4835

[P0009R9]
H. Carter Edwards, Bryce Adelstein Lelbach, Daniel Sunderland, D. S. Hollman, Christian Trott, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Mark Hoemmen. mdspan: A Non-Owning Multidimensional Array Reference. 20 January 2019. URL: https://wg21.link/p0009r9

[P1035R7]
Christopher Di Bella, Casey Carter, Corentin Jabot. Input Range Adaptors. 2 August 2019. URL: https://wg21.link/p1035r7

[P1255R4]
Steve Downey. A view of 0 or 1 elements: view::maybe. 17 June 2019. URL: https://wg21.link/p1255r4

[P1391R2]
Corentin Jabot. Range constructor for std::string_view. 17 June 2019. URL: https://wg21.link/p1391r2

[P1394R2]

Corentin Jabot, Casey Carter. Range constructor for std::span. 17 June 2019. URL: https://wg21.link/p1394r2

[P1629R1]

JeanHeyd Meneide. Transcoding the world - Standard Text Encoding. 2 March 2020. URL: https://wg21.link/p1629r1

[P1739R4]

Hannes Hauswedell. Avoid template bloat for safe_ranges in combination with 'subrange-y' view adaptors.. 1 March 2020. URL: https://wg21.link/p1739r4

[RANGE-V3]

Eric Niebler; Casey Carter. range-v3. June 11th, 2019. URL: https://github.com/ericniebler/range-v3